

Министерство образования Республики Беларусь
Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И
РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина Методы защиты информации

ОТЧЕТ
к лабораторной работе № 5
на тему
ХЕШ–ФУНКЦИИ

Выполнил

Т.П. Власенко

Проверил

А. В. Герчик

Минск 2024

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Результат выполнения программы	10
Заключение	11
Приложение А (обязательное) Исходный код программы	12

1 ПОСТАНОВКА ЗАДАЧИ

Цель выполнения данной лабораторной работы – реализовать самостоятельно (без использования готовых библиотек и функций) программное средство контроля целостности сообщений с помощью вычисления хеш-функции и алгоритма ГОСТ 34.11 и SHA 1

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Хеш-функции получили широкое распространение в разнообразных алгоритмах быстрого поиска информации. Однако с появлением криптографии у них появилась вторая, ничуть не меньшая, область применения.

Хеш-функцией (англ. hash – мелко измельчать и перемешивать) называется необратимое преобразование данных, обладающее следующими свойствами: на вход алгоритма преобразования может поступать двоичный блок данных произвольной длины; на выходе алгоритма получается двоичный блок данных фиксированной длины; значения на выходе алгоритма распределяются по равномерному закону по всему диапазону возможных результатов; при изменении хотя бы одного бита на входе алгоритма его выход значительно меняется: в идеальном случае инвертируется произвольная половина бит.

Основное, но не единственное, предназначение хеш-функций в криптографии – вычисление "неподделываемых" контрольных сумм документов.

Действительно, если для алгоритма хеширования выполняются требования невозможности подобрать иной документ с той же хеш-суммой и невозможности подобрать два документа с произвольной одинаковой хеш-суммой, то хеш-сумма становится уникальной характеристикой документа:

Алгоритм вычисления хеш-функции ГОСТ 34.11. В алгоритме ГОСТ 34.11 используются следующие преобразования:

Х-преобразование. На вход функции X подаются две последовательности длиной 512 бит каждая, выходом функции является XOR этих последовательностей.

S-преобразование. Функция S является обычной функцией подстановки. Каждый байт из 512-битной входной последовательности заменяется соответствующим байтом из таблицы подстановок π .

P-преобразование. Функция перестановки. Для каждой пары байт из входной последовательности происходит замена одного байта другим.

L-преобразование. Представляет собой умножение 64-битного входного вектора на бинарную матрицу A размерами 64×64 .

Функция сжатия. Основным элементом любой хеш-функции является функция сжатия. Опишем используемую в новом стандарте функцию сжатия g в виде алгоритма. Пусть h , N и m – 512-битные последовательности. Для вычисления функции $g(N, m, h)$ необходимо проделать следующие шаги:

1 Вычислить значение $K = h \oplus N$

2 Присвоить значение $K = S(K)$

3 Присвоить значение $K = P(K)$

4 Присвоить значение $K = L(K)$

5 Вычислить $t = E(K, m)$

6 Присвоить значение $t = h \oplus t$

7 Вычислить значение $G = t \oplus m$

Вернуть G в качестве результата вычисления функции $g(N, m, h)$

Функция $E(K, m)$ выполняет нижеприведенные действия:

1 Вычислить значение $state = K \oplus m$

2 Для $i=0$ по 11 выполнить:

– присвоить значение $state = S(state)$;

– присвоить значение $state = P(state)$;

– присвоить значение $state = L(state)$;

– вычислить $K = \text{KeySchedule}(K, i)$;

– присвоить значение $state = state \oplus K$.

3 Вернуть $state$ в качестве результата.

Функция $\text{KeySchedule}(K, i)$ отвечает за формирование временного ключа

K на каждом раунде функции $E(K, m)$ и производит следующие

вычисления:

1 Присвоить значение $K = K \oplus C[i]$.

2 Присвоить значение $K = S(K)$.

3 Присвоить значение $K = P(K)$.

4 Присвоить значение $K = L(K)$.

5 Вернуть K в качестве результата функции.

Вычисление хеш-функции. Для любого входного сообщения M :

1 Присвоить начальные значения внутренних переменных:

Для хеш-функции с длиной выхода 512 бит: $h=iv=0x00^{64}$

Для хеш-функции с длиной выхода 256 бит: $h=iv=0x01^{64}$

$N = 0^{512}$

$\Sigma = 0^{512}$

2 Проверить следующее условие: длина сообщения $M < 512$. Если условие выполняется перейти к пункту 3). В противном случае выполнить последовательность вычислений:

m – последние 512 бит сообщения M

$h = g(N, m, h)$

$$N = (N + 512) \bmod 2^{512}$$

$$\Sigma = (\Sigma + m) \bmod 2^{512}$$

Обрезать M, убрав последние 512 бит.

Перейти к шагу 2 .

3 Произвести дополнение сообщения M до длины в 512 бит по следующему правилу:

$$m = 0^{511-|M|} || 1 || M, \text{ где } |M| - \text{длина сообщения M в битах.}$$

Вычислить $h = g(N, m, h)$.

Вычислить $N = (N + |M|) \bmod 2^{512}$

Вычислить $\Sigma = (\Sigma + m) \bmod 2^{512}$

Вычислить $h = g(0, h, N)$

Вычислить $h = g(0, h, \Sigma)$

Для хеш-функции с длиной выхода в 512 бит возвращаем h в качестве результата. Для функции с длиной выхода 256 бит возвращаем MSB 256 (h).

Алгоритм вычисления хеш-функции SHA-1. Алгоритм SHA-1 (Secure Hash Algorithm) предложен Институтом Стандартизации США NIST как стандарт хеширования в гражданской криптографии. Этот алгоритм был призван дать еще больший запас прочности к криптоатакам.

SHA-1 реализует хеш-функцию, построенную на идее функции сжатия. Входами функции сжатия являются блок сообщения длиной 512 бит и выход предыдущего блока сообщения.

Выход представляет собой значение всех хеш-блоков до этого момента. Иными словами хеш блока M_i равен $h_i = f(M_i, h_{i-1})$. Хеш-значением всего сообщения является выход последнего блока. Алгоритм получает на входе сообщение максимальной длины 2^{64} бит и создает в качестве выхода дайджест сообщения длиной 160 бит.

Шаг 1: добавление недостающих битов

Сообщение добавляется таким образом, чтобы его длина была кратна 448 по модулю 512 (длина $\equiv 448 \bmod 512$). Добавление осуществляется всегда, даже если сообщение уже имеет нужную длину. Таким образом, число добавляемых битов находится в диапазоне от 1 до 512. Добавление состоит из единицы, за которой следует необходимое количество нулей.

Шаг 2: добавление длины

К сообщению добавляется блок из 64 битов. Этот блок трактуется как беззнаковое 64-битное целое и содержит длину исходного сообщения до добавления.

Результатом первых двух шагов является сообщение, длина которого кратна 512 битам. Расширенное сообщение может быть представлено как последовательность 512-битных блоков $Y_0, Y_1, \dots, Y_{L-1}$, так что общая длина расширенного сообщения есть $L * 512$ бит.

Таким образом, результат кратен шестнадцати 32-битным словам.

Шаг 3: инициализация SHA-1 буфера

Используется 160-битный буфер для хранения промежуточных и окончательных результатов *хеш-функции*.

Буфер может быть представлен как пять 32-битных регистров A, B, C, D и E.

Эти регистры инициализируются следующими шестнадцатеричными числами:

A = 67452301

B = EFCDAB89

C = 98BADCFE

D = 10325476

E = C3D2E1F0

Шаг 4: обработка сообщения в 512-битных (16-словных) блоках

Основой алгоритма является модуль, состоящий из 80 циклических обработок, обозначенный как H_{SHA} .

Все 80 циклических обработок имеют одинаковую структуру.

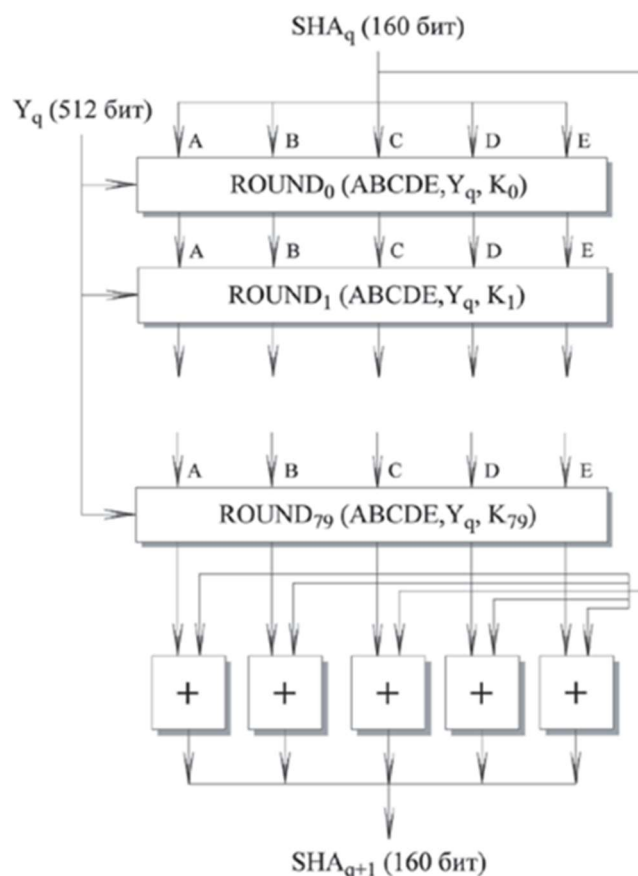


Рисунок 5,1 Обработка очередного 512-битного блока

Каждый цикл получает на входе текущий 512-битный обрабатываемый блок Y_q и 160-битное значение буфера $ABCDE$, и изменяет содержимое этого буфера.

В каждом цикле используется дополнительная константа K_t , которая принимает только четыре различных значения:

$$0 \leq t \leq 19 \quad K_t = 5A827999 \text{ (целая часть числа } [2^{30} \times 2^{1/2}])$$

$$20 \leq t \leq 39 \quad K_t = 6ED9EBA1 \text{ (целая часть числа } [2^{30} \times 3^{1/2}])$$

$$40 \leq t \leq 59 \quad K_t = 8F1BBCDC \text{ (целая часть числа } [2^{30} \times 5^{1/2}])$$

$$60 \leq t \leq 79 \quad K_t = CA62C1D6 \text{ (целая часть числа } [2^{30} \times 10^{1/2}])$$

Для получения SHA_{q+1} выход 80-го цикла складывается со значением SHA_q . Сложение по модулю 2^{32} выполняется независимо для каждого из пяти слов в буфере с каждым из соответствующих слов в SHA_q .

Шаг 5: выход

После обработки всех 512-битных блоков выходом L-ой стадии является 160-битный *дайджест сообщения*.

Алгоритм *SHA-1* можно суммировать следующим образом:

$$SHA_0 = IV$$

$$SHA_{q+1} = \Sigma_{32}(SHA_q, ABCDE_q),$$

Где:

IV - начальное значение буфера *ABCDE*;

$ABCDE_q$ - результат обработки q-того блока сообщения;

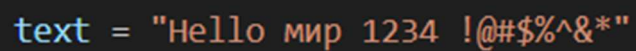
L - число блоков в сообщении, включая поля добавления и длины;

Σ_{32} - сумма по модулю 2^{32} , выполняемая отдельно для каждого слова буфера;

SHA - значение *дайджеста сообщения*.

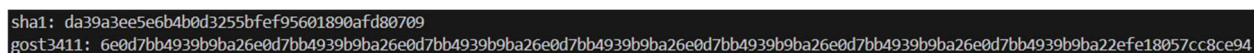
3 РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

На рисунке 3.1 представлен исходный текст, на рисунке 3.2 – результат вывода в консоль.



```
text = "Hello мир 1234 !@#$%^&*"
```

Рисунок 3.1 – Исходный текст



```
sha1: da39a3ee5e6b4b0d3255bfef95601890afd80709  
gost3411: 6e0d7bb4939b9ba26e0d7bb4939b9ba26e0d7bb4939b9ba26e0d7bb4939b9ba26e0d7bb4939b9ba26e0d7bb4939b9ba22efe18057cc8ce94
```

Рисунок 3.2 – Результат вывода в консоль

ЗАКЛЮЧЕНИЕ

В ходе данной лабораторной работы было самостоятельно реализовано (без использования готовых библиотек и функций) программное средство контроля целостности сообщений с помощью вычисления хеш-функции и алгоритма ГОСТ 34.11 и SHA 1

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

```
import numpy as np

pi = [i for i in range(512)]

A = np.random.randint(0, 100000000, size=(64, 64))

def X(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

def S(data):
    return bytes([pi[b] for b in data])

def P(data):
    permuted_data = bytearray(64)
    for i in range(0, 64, 2):
        permuted_data[i] = data[i + 1]
        permuted_data[i + 1] = data[i]

    return bytes(permuted_data)

def L(data):
    data_bits = np.unpackbits(np.frombuffer(data, dtype=np.uint8))
    result_bits = np.zeros(512, dtype=int)
    for i in range(8):
        block_bits = data_bits[i*64:(i+1)*64]
        result_bits[i*64:(i+1)*64] = np.dot(A, block_bits) % 2
    result = np.packbits(result_bits).tobytes()
    return result

def KeySchedule(K, i):
    C = [123, 51, 52, 11, 52, 13, 24, 245, 84, 11, 212, 112]
    K = X(K, bytes([C[i]] * 64))
    K = S(K)
    K = P(K)
    K = L(K)
    return K

def E(K, m):
    state = X(K, m)
    for i in range(12):
        state = S(state)
        state = P(state)
        state = L(state)
```

```

        K = KeySchedule(K, i)
        state = X(state, K)
    return state

def g(N, m, h):
    K = X(h, N)
    K = S(K)
    K = P(K)
    K = L(K)
    t = E(K, m)
    t = X(h, t)
    G = X(t, m)
    return G

def gost3411(M):
    M = text_to_bytes(M)

    h = bytes(64)
    N = bytes(64)
    Sigma = bytes(64)

    while len(M) >= 64:
        m = M[-64:]
        h = g(N, m, h)
        N = (int.from_bytes(N, 'big') + 512) % (2**512)
        N = N.to_bytes(64, 'big')
        Sigma = (int.from_bytes(Sigma, 'big') +
                  int.from_bytes(m, 'big')) % (2**512)
        Sigma = Sigma.to_bytes(64, 'big')
        M = M[:-64]

    m = b'\x00' * (63 - len(M)) + b'\x01' + M
    h = g(N, m, h)
    N = (int.from_bytes(N, 'big') + len(M) * 8) % (2**512)
    N = N.to_bytes(64, 'big')
    Sigma = (int.from_bytes(Sigma, 'big') +
              int.from_bytes(m, 'big')) % (2**512)
    Sigma = Sigma.to_bytes(64, 'big')
    h = g(bytes(64), h, N)
    h = g(bytes(64), h, Sigma)

    return h.hex()

def text_to_bytes(text):
    return text.encode('utf-8')
```